

WORLDLINE

Technical Whitepaper

Branchable working realities for Omarchy · engine 1.2.0 · 2026-09-20

Abstract

WORLDLINE turns a developer's working machine into a branchable reality. The state of nominated directories is captured as **PRIME**; PRIME is forked into isolated **worlds** where coding agents work without reaching the real files; each world is measured by evidence the project itself declares; and exactly one world is committed back through a single atomic filesystem exchange that a SPARK-proved kernel authorized. Refusal is the designed outcome whenever the facts do not support the commit. This paper states the problem, the layering, the identity and evidence model, what the kernel proves and what it does not, the isolation boundary (including the network containment added in 1.2.0), the transaction protocol, the receipt chain and its external anchor, and the operating limits a finished tool carries.

1. The problem

Coding agents are cheap to run and expensive to trust. Running one against a live checkout means every mistake is a mistake in the real tree, and comparing two approaches means either two clones drifting apart by hand or serial trial. The usual tools cover part of this: version control records what was committed, containers isolate processes, sandboxes restrict syscalls. None of them answers the working question: *let three agents try this, show me what each one actually did and whether it passed my checks, and put exactly one of them in, atomically, with a receipt I can verify later.*

1.1 Why existing tools do not cover it

Git branches share a working tree and cannot host three concurrent agents on the same paths. Worktrees can, but they carry no evidence, no atomic merge into a live tree, and no notion of generated state. Containers isolate but do not compare. A sandbox limits damage but produces no receipt. WORLDLINE composes these: overlayfs and bubblewrap for isolation, manifests and hashes for identity, declared checks for evidence, a proved decision procedure for authorization, and `renameat2(RENAM_EXCHANGE)` for the commit.

2. Design principles

1. **A world is a proposal; PRIME is the only reality.** Nothing becomes real except through an authorized collapse.
2. **Every claim is observed or labelled.** States, evidence, risk, and provenance carry their inputs; missing evidence is UNASSESSED, a missing capability is UNAVAILABLE with a reason.

3. **Refusal is an answer.** Every refusal has a code; the runtime never routes around one.
4. **The proof boundary is stated, not blurred.** What the kernel proves is listed; what sits outside it (the C/Python/QML boundary, the OS, the filesystem) is listed next to it.
5. **Containment claims are calibrated.** Files are contained by construction; the network is contained only under the policy that says so.

3. Architecture

3.1 Layering and responsibilities

Layer	Language	Responsibility
Kernel <code>libworldline_core.so</code>	Ada 2022 / SPARK	Hashing (SHA-256 via ATTEST), world identity, causal and receipt links, the world and transaction state machines, the collapse decision
Runtime	Python 3.14	Capture, manifests, deltas and three-way merge, sandboxing, supervision, transactions, receipts, anchors, pruning, the daemon and CLI
Desktop	QML (Quickshell)	The cockpit: fork, multiverse graph, review-and-commit, roots, diagnostics; a bar widget

The kernel is deliberately small and pure: it computes and compares; it never touches the filesystem. The runtime decides *what* is compared and performs every effect. This division is what makes the proof meaningful: the proved part is the decision, and the decision's inputs are hashes the runtime is obliged to compute honestly.

3.2 Process, privilege, and storage model

One user daemon (`worldlined`, a `systemd` user unit) owns the store and the socket; a singleton lock refuses a second instance (`DAEMON_ALREADY_RUNNING`). Clients speak NDJSON over an owner-only Unix socket with a peer-credential check per connection. Every world runs as a transient `systemd` unit inside a bubblewrap sandbox; the daemon supervises it, and a daemon restart sweeps unsupervised worlds to a terminal state through the kernel (`DAEMON_RESTART`).

Storage lives under the user's XDG directories: generations and world payloads (data), the SQLite store, logs, receipts, and the anchor ledger (state), the socket and the published status document (runtime). The SQLite schema is versioned (`user_version`) with forward-only, recorded migrations; a store newer than the runtime is refused rather than opened.

4. Identity and capture

4.1 Canonical encoding

Every document that participates in an identity — manifests, events, receipts — is serialized canonically (sorted keys, no floats, fixed escapes) and hashed by the kernel. Two documents with the same meaning have the same bytes, so equality of hashes is equality of content.

4.2 Manifest capture

A manifest lists every entry under a root with its type, mode, size, content hash, symlink target, and hard-link group, plus root metadata and, for `repo` roots, the repository facts (head, branch, index hash, status and diff hashes). Symlinks escaping the root, special files, and cross-device entries are refused at capture. A root set hash binds the manifests of all registered roots; a world's content identity binds filesystem, config, repository, environment, and evidence components to its parent.

Since 1.2.0, repository inspection runs against a private copy of the index. Git refreshes and rewrites the index when stat data looks racy, which a freshly materialized copy always does; that write changed the staged tree between prepare and commit and made every repository-root collapse refuse. Capture is now byte-neutral and deterministic, and the git cycle (agent commits in its world, collapse lands the commit, return removes it) is a boundary test.

5. Evidence

A project declares its checks in `.worldline.json`: build, tests, proofs, benchmarks, health, each with a format the runtime can parse and a `required` flag. Checks run inside the world's sandbox after the agent finishes. The world's evidence is the list of results; its state is `VALID` only if the agent exited successfully and every required check passed. Risk and complexity are derived labels printed next to their inputs. A world that was cancelled or timed out reports its checks as `UNASSESSED` with the reason: running a test suite against a half-finished tree would manufacture evidence.

Under the `allowlist` network policy the agent check also carries a network block: the allowed hosts, the number of connections, and every refused host with a count. A refusal is evidence of what the agent wanted.

6. The proved kernel

6.1 Proved units and exported interface

The kernel exports a C ABI: `wl_hash_bytes`, `wl_hash_file`, `wl_world_id`, `wl_causal_link`, `wl_receipt_link`, `wl_transition_allowed`, `wl_transaction_transition_allowed`, and `wl_collapse_decide`. The SPARK units carry proofs of absence of runtime errors and of the policy properties: the world state machine admits only the documented transitions; the transaction state machine admits `PREPARED` → `AUTHORIZED`

→ COMMITTED and the abort/deny paths and nothing else; `collapse_decide` returns AUTHORIZED only when the candidate is VALID, has no conflicts and no foreign managed writes, and every identity the runtime hands it (parent, owner, base, delta, root set, staged root) matches its expectation.

6.2 Collapse authorization

Prepare computes, from fresh captures: the base manifests (the checkpoint the candidate was forked from), the current manifests (PRIME as it is now), and the candidate manifests; performs a three-way merge into a staged payload; records conflicts and contamination; and hashes every input. Commit recaptures PRIME and the staged payload, refuses if either moved (PRIME_CHANGED_AFTER_PREPARE, STAGED_ROOT_MISMATCH), and only then asks the kernel. The kernel's parent check compares the parent identity the *store* knows against the candidate's claim; feeding the claim to both sides, as 1.0 did, made the check tautological.

6.3 The proof gate

`prove.sh` runs GNATprove at level 3 over every kernel unit and fails unless every check is proved with nothing assumed and nothing justified (130 checks as of 1.2.0). It writes `proof-manifest.json` with the library hash; the installer verifies the manifest before installing, and the runtime reports `invariantPreservation: PROVED` in a receipt only while the running library still matches. The continuous-integration workflow re-proves every build on an x86_64 runner, which is also the portability check for the Mac Pro.

6.4 What is not proved

The C/Python/QML boundary, the operating system, overlayfs, bubblewrap, systemd, SQLite, and the filesystem's behaviour under crash are outside the proof. Receipts list this under `boundary.notProved`. The runtime's honesty about its inputs is enforced by tests, fault injection (disk full, `kill -9` with a prepared transaction, hostile root contents), and the boundary suite, not by proof.

7. Isolation

7.1 Mount policy

A world sees `/usr`, `/bin`, `/sbin`, `/lib`, `/lib64`, `/etc`, `/sys`, `/var`, `/opt` read-only; a fresh `/run`, `/tmp`, and `/var/tmp`; the operator's home as an empty tmpfs; the declared read-only home paths (toolchains) and the adapter's credential file bound read-only at their real paths; each managed root as an overlay at its original absolute path with the frozen checkpoint as the lower layer; and its own runtime directory at `/run/worldline-runtime`. Host live symlinks, Wayland and D-Bus sockets, and WORLDLINE's own store are absent. The resolver directory of systemd-resolved is bound read-only so `/etc/resolv.conf` resolves (without it, every agent failed DNS). Agent worlds run

as the real uid inside the user namespace; system futures keep namespace root; both are neutered (--cap-drop ALL, --disable-usersns, NoNewPrivileges).

7.2 Credentials never become state

Credential files are projected, never copied into a payload, and never captured into a manifest. The one exception is by construction: omp keeps credentials and state in one SQLite file it opens read-write, so a world gets a per-world private copy made through the SQLite backup API; the host file is never mounted.

7.3 Network containment (1.2.0)

Under `network.policy: shared` the world shares the host network namespace, which is what lets an agent reach its model and is exactly what the isolation claim did not cover. Under `allowlist` the world gets an empty network namespace (--unshare-net, loopback only) and one door: the daemon runs an HTTP proxy on a Unix socket bound into the world's runtime directory; a forwarder inside the world listens on loopback, relays to that socket, and starts the agent with the proxy environment set. The proxy connects only to the adapter's provider hosts plus `network.allow`; everything else is answered 403 and recorded on the world's evidence. `none` is the same namespace without the door. Checks and services get no network under either restrictive policy. Verified for real: a Claude Code world completed its mission through the proxy while its attempts to reach GitHub, telemetry, and the remote MCP proxy were refused and listed.

7.4 Supervision, timeouts, and finalization

Each world is a transient unit the daemon waits on. `cancel` stops the unit; --timeout (or a configured default) stops it on expiry. Either way the world finalizes honestly: the agent check fails with the reason, project checks are UNASSESSED, the job state says CANCELLED or TIMED_OUT, and the partial payload is materialized and inspectable. Finalization copies the world's overlay into its payload inside a network-less sandbox, captures manifests, runs checks, computes the delta against the base checkpoint, and moves the world to VALID or DEGRADED through the kernel.

8. Transactions

8.1 Three-way staging and conflicts

Base, current, and candidate manifests drive a three-way merge into a staged payload: a path the candidate changed and PRIME did not is taken from the candidate; a path both changed differently is a conflict; a path changed in PRIME by something other than a WORLDLINE commit is contamination. Conflicts and contamination are facts on the transaction, and the kernel denies a collapse that has either.

8.2 Prepare, commit, recover

A transaction is a durable record: `PREPARED` with every hash and the staged payload, `AUTHORIZED` once the kernel agreed, `COMMITTED` once the exchange landed. Commit fsyncs the staged tree, then performs one `renameat2(RENAME_EXCHANGE)` on the live mapping. Recovery after a crash re-hashes survivors and either completes a committed exchange or aborts a prepared one; a transaction it cannot resolve is quarantined and blocks mutation while diagnosis stays available. The cockpit uses `--prepare` and `transaction commit` so the facts a human reviewed are the facts that commit.

8.3 Return

`return` restores the state a checkpoint had at the instant it was displaced. A checkpoint that was live for weeks accumulates generated artifacts its stored manifest predates; it is verified against the receipt whose `beforeRoot` recorded the pre-exchange state, or against the state root of the reconcile checkpoint published from it. A checkpoint that changed after it left reality matches neither and is refused. This was found on the live machine after the first real collapse and is now a boundary test.

8.4 Collapse receipts and the anchor

Every commit produces a receipt: transaction, parent and candidate identities, `beforeRoot` and `afterRoot`, the delta, contamination, generated paths, dependency changes, the proof claim, and the non-claims, chained by hash to the previous receipt. `log --verify` replays the chain through the kernel.

The receipt chain inside the store is self-consistent and that is all it is: a process running as the operator can rewrite the store and the chain still verifies. Since 1.2.0 every receipt is also appended to an Ed25519-signed, hash-chained ledger written in the Custos format, so `attest verify-custos` — a separate SPARK-proved program with its own SHA-256 and Ed25519 — replays the chain and every signature using none of WORLDLINE's code. The ledger is mirrored to `anchor.exportPath`; verification compares heads and distinguishes a matching copy, an export that fell behind, a local rollback, and a mismatch. The residual assumption is stated: an attacker who holds the signing key and can reach the export location is outside this defence.

9. Retention

Every fork materializes a full checkpoint and every finished world keeps its payload. `prune` reclaims the payload directories of finished worlds nothing living refers to — never `PRIME`, the default return point, a running world, or a directory still referenced by an unpruned world or a live root — marks the worlds pruned, and records a causal event per world. Records and receipts are permanent; a pruned world refuses inspection and return by name. Store usage by area is part of `doctor`.

10. Desktop integration

The Omarchy plugin renders the published status document (nine fixed fields; a world's file list is capped and says so) and drives every mutation through the CLI's `--prepare` and `transaction` contract. The evidence vocabulary (PASS, FAIL, UNASSESSED, UNAVAILABLE, STALE) is derived in one place and shared by the bar and the overlay. Diagnostics render `doctor`, including the anchor verdicts, store usage, the network policy, and the default timeout.

11. Verification record

Engine 1.2.0 ships with 94 runtime tests, the Ada behaviour and fuzz executables, and a proof gate of 130 proved checks. Beyond the suite, the following were exercised for real on the reference machine: all builtin adapters with the operator's credentials; a three-lane race; collapse and return of a real world in a private harness and on the live PRIME; a git repository root cycle; the allowlist policy with a real Claude Code world; a real timeout; four real ghost worlds on one checkpoint; prune; the anchor ledger verified by `attest` with a rollback drill; disk-full injection; `kill -9` with a running agent and a prepared transaction; and the installer's rollback procedure.

Appendix A. Canonical status document

`schemaVersion`, `daemon`, `prime`, `activeWorld`, `worlds`, `jobs`, `capabilities`, `lastReceipt`, `ghostRecommendation`. Publication fails rather than drifting from that shape.

Appendix B. Identity domains

`worldline-manifest-v1`, `worldline-return-v1`, `attest-entry-v1` (kernel hashing domains), `custos-v1` and `custos-genesis-v1` (anchor ledger, shared with ATTEST).